Lab: Similarity Search on Text Using a Chroma Vector Database

Estimated time: 30 minutes

What you will learn

In this lab, you will work with the implementation of a text similarity search using embeddings in Chroma DB, a database system designed for managing and querying text data efficiently.

You will learn to create a collection and populate the database with a set of sample text documents, where each document is associated with a unique ID and its corresponding embedding generated using the default text embedding function provided by Chroma DB. Subsequently, you will also perform a similarity search on a specified query term, retrieving the top three documents that exhibit the highest similarity to the query term.

Learning objectives

After completing this lab, you will be able to:

- Integrate Chroma DB into applications for the storage and querying of text data.
- Generate text embeddings for text documents using Chroma DB's default embedding function.
- Implement a similarity search functionality to compute similarity scores between query embeddings and document embeddings
- Display top-ranked documents similar to a given query term.

About Skills Network Cloud IDE

Skills Network Cloud IDE (based on Theia and Docker) provides an environment for hands on labs for course and project related labs. It is an open source IDE (Integrated Development Environment).

Important Notice about this lab environment

Please be aware that sessions for this lab environment are not persisted. Every time you connect to this lab, a new environment is created for you. Any data you may have saved in the earlier session would get lost. Plan to complete these labs in a single session, to avoid losing your data.

Prerequisites

- Basic knowledge of vector databases
- An understanding of Chroma vector databases
- Use the Setting up a Chroma DB environment lab before completing this lab's tasks. Remember to keep your terminal window open.

Task 1: Setting up a Chroma DB environment

1. On the main menu, select the **Terminal** tab and then select **New Terminal**.

2. In the terminal run the Docker command to pull a Chroma DB image to work with the database.

   ```
   docker run -p 8000:8000 chromadb/chroma
   ```

   Important! Please keep this terminal window open, as Docker execution will be required until the end of the lab.

3. Now open new terminal and install `chromadb` in the environment to work with the Chroma DB vector database. Perform the following command in the terminal window and press **Enter**.

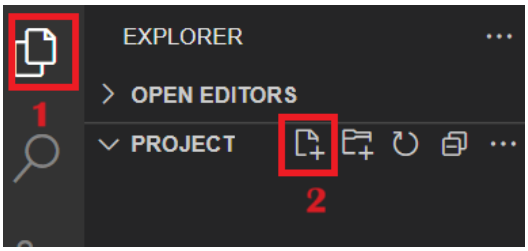
   ```
   npm install chromadb@1.9.1
   ```

4. Then install one more dependency by performing the following command in the same terminal window where you installed the previous dependency.

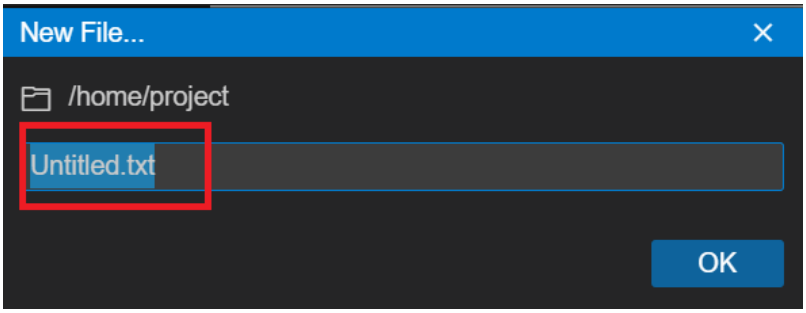
```
npm install chromadb-default-embed
```

Note: Do not close the terminal where Docker is running.

5. Select **Explorer** on the left side of the terminal window, as shown at number 1 in the following screenshot. Then, within the **Project** folder, select **New File** as shown at number 2 in the following screenshot.



6. After performing above step, a popup box displays with default file name **Untitled.txt** as shown in the following screenshot. Replace it's default name with `similaritySearch.js`.



Task 2: Create a Collection and Embed Data

Now, you will learn how to create a collection and embed data.

Create a Collection and Embed Data

1. First, import the ChromaClient class from the `chromadb` package. By importing the ChromaClient class, you can create an instance of the client that interacts with the Chroma DB database. To import the class, use the `require` keyword. Include the following command in your JavaScript file.

```
// Importing the necessary classes from the chromadb package:
// ChromaClient is used to interact with the Chroma DB database,
// DefaultEmbeddingFunction is used to handle embeddings (optional depending on use case)
const { ChromaClient, DefaultEmbeddingFunction } = require('chromadb');
// Create a new instance of ChromaClient to interact with the Chroma DB
const client = new ChromaClient();
```

The code imports necessary modules from Chroma DB, including ChromaClient for interacting with the database and `DefaultEmbeddingFunction` to generate embeddings from text data.

2. Then initialize the `collectionName` variable and specify the name of the collection. Also, initialize one instance, which you will name `DefaultEmbeddingFunction`.

```
// Define the name for the collection to be created or retrieved
const collectionName = "my_grocery_collection";
// Create an instance of DefaultEmbeddingFunction (optional, depending on how you plan to use embeddings)
const default_emd = new DefaultEmbeddingFunction();
```

3. Create the main function, which you must define as an asynchronous function. This function will contain the code for interacting with the Chroma DB database.

```
// Define the main asynchronous function to interact with the Chroma DB
async function main() {
try {
    // Place your asynchronous code (e.g., database operations) inside this block
  } catch (error) { // Catch any errors and log them to the console
    console.error("Error:", error);
  }
}
```

4. Then create a collection inside of the database and perform the following command inside the `try` block of `main` function.

```
// Retrieve or create a collection in the Chroma database with a specified name and embedding function
const collection = await client.getOrCreateCollection({
        name: collectionName, // The name of the collection you want to retrieve or create
        embeddings: default_emd // The default embedding function used to process data
    });
```

- This code retrieves or creates a collection named "my_grocery_collection" from the database using the client's `getOrCreateCollection` method.
- This method provides a way to ensure that a collection exists in the database, either by retrieving an existing collection or by creating a new collection if a collection doesn't already exist.
- The collection variable holds a reference to this collection for further operations and is also configured to use the default embedding function to store embeddings with text data.

5. Next, after creating a collection inside the try block, define the sample data. The following array exists as text and contains the sample text strings for fruit.

```
// Array of grocery-related text items (e.g., product descriptions or names)
const texts = [
            'fresh red apples',
            'organic bananas',
            'ripe mangoes',
            'whole wheat bread',
            'farm-fresh eggs',
            'natural yogurt',
            'frozen vegetables',
            'grass-fed beef',
            'free-range chicken',
            'fresh salmon fillet',
            'aromatic coffee beans',
            'pure honey',
            'golden apple',
            'red fruit'
        ];
```

6. Now, generate unique IDs for each text string in the array by including the following command after the text array using the following code:

```
// Create an array of unique IDs for each text item in the 'texts' array
 // Each ID follows the format 'food_<index>', where <index> starts from 1
const ids = texts.map((_, index) => `food_${index + 1}`);
```

- This code generated unique IDs for the documents based on their index in the `texts` array. Each ID applies the format `"food_<index>`“, where the `<index>` starts with the number 1.

Task 3: Generate Embeddings and Add Texts to a Collection

1. Next, generate embeddings for each index value of the texts array using the following code:

```
// Generate embeddings for the texts using the default embedding function
// The `generate` method processes the texts and returns their corresponding embeddings
const embeddingsData = await default_emd.generate(texts);
```

- The `default_ef.generate(texts)` function calls generate embeddings for the given texts array where embeddings are numerical representations of text data. The texts array contains a list of sample text data, such as "fresh red apples", "organic bananas", and so on.
- The generate method of the `DefaultEmbeddingFunction` class generates embeddings for each text in the array using a pre-defined embedding function. These embeddings are then stored in the `embeddingsData` variable.

2. Then add the IDs and texts to a collection using the following code:

```
// Add documents, their corresponding IDs, and embeddings to the collection
// The `add` method inserts the data into the "groceries" collection
// The documents are the actual text items, the IDs are unique identifiers, and the embeddings are numeric vector representations of the text
await collection.add({ ids: ids, documents: texts, embeddings: embeddingsData });
```

- By passing these three pieces of information using the `add` method of the collection, the code adds the documents, along with their corresponding IDs and embeddings, to the Chroma DB database collection.
- This code supports efficient storage and retrieval of text data, along with their embeddings, to facilitate natural language processing tasks such as similarity search and document clustering.

3. You can use the following code, which uses the `get` method, to access all items.

```
// Retrieve all the items (documents) stored in the collection
// The `get` method fetches all data from the collection
const allItems = await collection.get();
// Log the retrieved items to the console for inspection
// This will print out all the documents, IDs, and embeddings stored in the collection
console.log(allItems);
```

- `const allItems` This line of code declares a constant variable named allItems to store the result of the collection.get() method. This variable holds the retrieved items from the collection, including the documents, IDs, and embeddings.
- `collection.get()` This `get` method retrieves all items stored within the collection. These items include the documents, their corresponding IDs, and their embeddings.
- `await` The await keyword is used to wait for the asynchronous `collection.get()` method to complete its execution before moving to the next line of code.

Task 4: Perform a Similarity Search

1. Use the following code to create one function named `performSimilaritySearch` after the main function. You will write your query using similar items you want to identify.

```
// Asynchronous function to perform a similarity search in the collection
async function performSimilaritySearch(collection, allItems) {
}
```

2. In this function write the "try and catch" block to catch the errors easily. Within the "try" block intialize one variable named `query`. Here's the example code:

```
// Define the query term you want to search for in the collection
 const queryTerm = "apple";
```

3. Then, create the code that calls the `query` method. The query method calls on the collection to search for documents similar to the query term, which also includes the *n* parameter. The *n* parameter specifies the number of top results to retrieve.

```
// Perform a query to search for the most similar documents to the 'queryTerm'
const results = await collection.query({
            collection: collectionName,
            queryTexts: [queryTerm],
            n: 3 // Retrieve top 3 results
        });
console.log(results);
```

Task 5: Handle the Results and Display Highly Similar Text

1. Next, create code, using the following example code, that handles the results if no similar documents to the query term are found and logs a message to the console.

```
// Check if no results are returned or if the results array is empty
if (!results || results.length === 0) {
    // Log a message indicating that no similar documents were found for the query term
    console.log(`No documents found similar to "${queryTerm}"`);
return;
}
```

2. You'll also want to display the top documents whose distance is less from the query than other text data.

```
console.log(`Top 3 similar documents to "${queryTerm}":`);
// Access the nested arrays in 'results.ids' and 'results.distances'
    for (let i = 0; i < 3; i++) {
        const id = results.ids[0][i]; // Get ID from 'ids' array
        const score = results.distances[0][i]; // Get score from 'distances' array
// Retrieve text data using the document ID
    const text = allItems.documents[allItems.ids.indexOf(id)];
    if (!text) {
        console.log(` - ID: ${id}, Text: 'Text not available', Score: ${score}`);
    } else {
    console.log(` - ID: ${id}, Text: '${text}', Score: ${score}`);
    }
    }
```

3. Next, call the `performSimilaritySearch` function at the end of the `try` block of the `main` function using the following code:

```
await performSimilaritySearch(collection, allItems);
```

4. Then call the `main` function to create a collection and generate embeddings.

```
main();
```

Click here to view the full solution code.

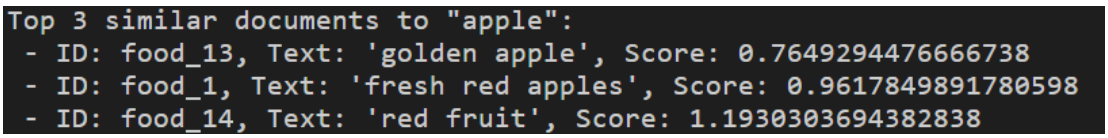
Task 6: Check the output

Next, check the output so that you can verify that the output displays show the most similar items or data in the text array.

1. If the terminal window is not already open, open the terminal window. Then type or enter the following command and select **Enter**.

```
node similaritySearch.js
```

2. The output will display similarl to what is seen in the following screenshot where you see your top three documents similar to apple and see their IDs, text, ans similarity scores.



Congratulations! You have created your first application of similarity search!

Practice Exercises

In this lab session you generated embeddings for `texts` arrays and performed similarity search using the `const queryTerm="red"` query string. In this set of practice exercises you will go through the steps needed to perform similarity search on the same `texts` array but using a different query string such as`const queryTerm=["red","fresh"]`.

1. Replace or comment `const queryTerm="red"`with `const queryTerm=["red","fresh"]` in your current code file.

   Hint: Put two forward slashes `//` in front of the query term with "red" as the value and add a mew line that specifies new values for the query string.

   Hint: Consider the terms "eggs" and "fresh".

▶ Click here for the solution

2. Pass the `queryTerm`in the form of single string by concatenating the array elemnts using the `join()` method within the `performSimilaritySearch()` function. Include this information after the code in practice exercise 1.

   Hint: Use the `join()` method to combine elements as strings.

▶ Click here for the solution

3. Within the result variable of current code pass the `queryTerm` to perform similarity search.

Hint: Use `query()` method

▶ Click here for the solution

5. Verify that the output is correct by running the current file name in the terminal using `node` with file name `similaritySearch.js`.

```
node similaritySearch.js
```

Now that you've completed each of the practice exercises, click below to see the entire code flow of `similaritySearch.js`.

Click here to view the full solution code.

Conclusion

In this lab, you learned that:

- **Chroma DB integration:** The code demonstrates the integration of Chroma DB with a library for similarity search and embeddings. The code imports necessary modules from Chroma DB create a client instance and defines a default embedding function.
- **Data handling:** The code creates a collection named "my_grocery_collection" within the Chroma DB client. The code defines sample text documents related to food items and adds these text documents to this collection along with their corresponding IDs. The default embedding function generates embeddings for the text documents and adds those documents to the collection.
- **Similarity search:** The code defines a function named "performSimilaritySearch" to run a similarity search based on a specified query term. In this case, the code specifies the query term "apple" to retrieve the top three similar text documents from the collection. The function iterates through the results and retrieves the results that match the top three texts with the same query parameter.

Author(s)

Richa Arora